Smart Security System — Serial Monitor Demo Run Walkthrough

This guide walks through a complete live demo of the Smart Security System from power-up through

every major state transition. It is intended for use when demonstrating the system to others for

the first time. Follow the phases in order, watch the Serial Monitor at 9600 baud alongside the

hardware, and you will see every output described below.

Hardware Checklist Before Starting

- Arduino Uno R3 connected via USB
- HC-SR04 ultrasonic sensor wired to pins 5 (TRIG) and 6 (ECHO)
- Fasizi 1.8" TFT LCD wired via SPI (CS→10, DC→9, RST→8, MOSI→11, SCK→13)
- Red LED on pin 3 (220 Ω series resistor), Green LED on pin 4 (220 Ω series resistor)
- Active buzzer on pin 7
- Push button on pin 2 (INPUT_PULLUP — connects pin 2 to GND when pressed)
- Arduino IDE Serial Monitor open, baud rate set to **9600**

Phase 1 — Power-Up

What happens: The Arduino runs `setup()`. Serial is initialised, all pin modes are

configured, the TFT is initialised via `initR(INITR_BLACKTAB)`, and `enterState(DISARMED)` is

called automatically.

What you see on the hardware:

- TFT clears to black, then displays **"SYSTEM DISARMED"** in green text

- Green LED turns **on**

- Red LED stays **off**

- Buzzer stays **off**

Serial Monitor output:

```
[STATE] DISARMED -> DISARMED
```

> The initial call to `enterState(DISARMED)` logs a transition from the uninitialised state

> to DISARMED. Everything is quiet — the system is waiting for a button press.

Phase 2 — DISARMED Display

What happens: The system sits in the DISARMED state. No sensor polling occurs. The loop

runs continuously but only `checkButton()` does meaningful work — it watches for a falling

edge on pin 2.

What you see on the hardware:

- TFT shows `SYSTEM` (line 1) and `DISARMED` (line 2) in **green** on a black background

- Green LED is on continuously

- No distance readings appear — the HC-SR04 is not triggered in this state

Serial Monitor output:

```
(no output — system is waiting)
```

> *This silence is correct behaviour. The sensor is intentionally not polled while disarmed*

> *(Requirement 3.5), so no `[DIST]` lines appear here.*

Phase 3 — Button Press to ARMED

What happens: Press the push button once. `checkButton()` detects the falling edge (pin 2

goes HIGH → LOW), confirms that at least 50 ms has elapsed since the last registered press, and

calls `handleButtonPress()`. The state machine transitions DISARMED → ARMED.

What you see on the hardware:

- TFT clears and shows **"SYSTEM ARMED"** in red, followed by `Dist: -- cm` on the lower line

- Green LED turns **off**

- Red LED turns **on** (continuous)

- Buzzer stays off

Serial Monitor output:

```
[BTN] Button pressed
[STATE] DISARMED -> ARMED
```

> *The button confirmation always appears before the state transition log because*
> *`handleButtonPress()` prints `[BTN]` first, then calls `enterState()`.*

Phase 4 — ARMED — No Object Detected

What happens: With nothing in front of the sensor, `updateSensor()` fires every 200 ms and

logs distance to Serial every 500 ms. Distances well above 50 cm keep the grace period reset.

What you see on the hardware:

- TFT distance line refreshes roughly 5× per second, showing the live reading
- Red LED stays on continuously

Serial Monitor output (typical open-air readings):

```
[DIST] 182.3 cm
```

[DIST] 181.9 cm

[DIST] 183.1 cm

[DIST] 182.7 cm

```

*> Readings vary slightly — this is normal ultrasonic behaviour. As long as every value is*

*> above 50.0 cm, the grace period remains inactive and no `[GRACE]` lines appear.*

---

**## Phase 5 — Object Placed Within 50 cm — Grace Period Starts**

**What happens:** Move your hand (or any solid object) to within 50 cm of the HC-SR04. On the

first reading at or below 50.0 cm, `updateSensor()` sets `graceActive = true` and records

`graceStartMs`. `updateStateMachine()` then starts printing elapsed grace period time on every

subsequent loop iteration where `graceActive` is true.

**What you see on the hardware:**

- TFT distance line drops to the measured value (e.g. `Dist: 38.4 cm`)

- Red LED stays on

**Serial Monitor output:**

```

[DIST] 38.4 cm

[GRACE] Elapsed: 0 ms

[GRACE] Elapsed: 4 ms

[GRACE] Elapsed: 8 ms

[GRACE] Elapsed: 12 ms

[DIST] 37.9 cm

[GRACE] Elapsed: 204 ms

[GRACE] Elapsed: 208 ms

```

> `[GRACE]` lines are printed every loop iteration when `graceActive` is true — they will

> scroll quickly. `[DIST]` lines continue every 500 ms interleaved with the grace output.

---

**## Phase 6 — Grace Period Countdown to 3 Seconds**

**What happens:** Keep the object in place. The elapsed time ticks upward. As long as every

sensor reading remains ≤ 50 cm, the grace window stays open. If the object is briefly removed

(reading > 50 cm or a timeout), the timer resets to zero and a fresh 3-second window must

start from the next detection.

**Serial Monitor output (object held steady for ~3 seconds):**

```

[DIST] 38.2 cm

[GRACE] Elapsed: 500 ms

[GRACE] Elapsed: 504 ms

...

[DIST] 37.8 cm

[GRACE] Elapsed: 1000 ms

[GRACE] Elapsed: 1004 ms

...

[DIST] 38.5 cm

[GRACE] Elapsed: 1500 ms

...

[DIST] 38.1 cm

[GRACE] Elapsed: 2000 ms

...

[DIST] 37.6 cm

[GRACE] Elapsed: 2500 ms

...

[DIST] 38.3 cm

[GRACE] Elapsed: 2900 ms

[GRACE] Elapsed: 2904 ms

[GRACE] Elapsed: 2908 ms

```

*> If the object is momentarily removed you will see distance jump above 50 cm, the*

*> `[GRACE]` lines stop, and when it returns a fresh `[GRACE] Elapsed: 0 ms` sequence begins.*

---

**## Phase 7 — ALARM Triggers at 3 000 ms**
```

What happens: When `elapsed >= 3000 ms`, `updateStateMachine()` calls `enterState(ALARM)`.

The buzzer fires immediately, timers for the LED blink and TFT flash are seeded, and the display

changes.

What you see on the hardware:

- TFT clears and shows **"! INTRUDER ALERT!"** — starts in red, flips to yellow after 500 ms,

then back to red, cycling continuously

- Red LED starts **blinking** (250 ms on / 250 ms off)

- Buzzer produces a **siren** cadence: 300 ms on, 200 ms off, repeating

- Green LED stays off

Serial Monitor output:

```
[GRACE] Elapsed: 3000 ms
[STATE] ARMED -> ALARM
```

> Output goes quiet after the transition log — there are no periodic Serial prints defined

> for the ALARM state itself. The hardware is doing the work (buzzer, LEDs, TFT flash).

Phase 8 — ALARM Phase (30-Second Window)

What happens: The system sits in ALARM. `updateAlarmPatterns()` drives three independent

non-blocking timers. The 30-second auto-silence timer (`alarmEnteredMs`) runs in parallel.

What you see on the hardware (ongoing for ~30 seconds):

- TFT alternates between red and yellow "! INTRUDER ALERT!" every 500 ms

- Red LED blinks every 250 ms

- Buzzer cycles: 300 ms on → 200 ms off → 300 ms on → ...

Serial Monitor output:

```
(no further output until auto-silence fires)
```

> *Demonstrating the hardware patterns to your audience during this 30-second window is the*

> *highlight of the demo. Point out the asymmetric buzzer cadence (300 ms on vs 200 ms off)*

> *which gives it the siren character rather than a simple beep.*

Phase 9 — 30-Second Auto-Silence — ALARM → ARMED

What happens: When `millis() - alarmEnteredMs >= 30 000 ms`, `updateStateMachine()` calls

`enterState(ARMED)`. All alarm outputs are cleared and the system returns to active monitoring.

What you see on the hardware:

- Buzzer turns **off**

- Red LED switches from blinking to **steady on**

- TFT clears and shows **"SYSTEM ARMED"** in red with `Dist: -- cm` (live reading resumes

within 200 ms)

- Green LED stays off

Serial Monitor output:

```
[STATE] ALARM -> ARMED

[DIST] 43.2 cm

[DIST] 43.0 cm
```

> *The system is now armed again and will re-trigger the alarm if an object comes within 50 cm*

> *for another continuous 3 seconds. This demonstrates Requirement 7 — fully autonomous recovery.*

Phase 10 — Manual Disarm from ARMED

What happens: Press the push button once while in ARMED.
`handleButtonPress()` transitions

ARMED → DISARMED. This can be done at any point during the ARMED state (or from ALARM, which

also goes to DISARMED directly).

What you see on the hardware:

- TFT clears and shows **"SYSTEM DISARMED"** in green

- Red LED turns **off**

- Green LED turns **on**

- Buzzer stays off

- Distance readings stop

Serial Monitor output:

```
[BTN] Button pressed

[STATE] ARMED -> DISARMED
```

> The system is now back to Phase 2. Press the button again to re-arm and repeat any part

> of the demo.

Quick Reference — Serial Output Tags

| Tag | When it appears | Example |

|---|---|---|

| `[BTN]` | Every confirmed (debounced) button press | `[BTN] Button pressed` |

| `[STATE]` | Every state transition | `[STATE] ARMED -> ALARM` |

| `[DIST]` | Every 500 ms while in ARMED state | `[DIST] 38.4 cm` |

| `[GRACE]` | Every loop iteration while grace period is active | `[GRACE] Elapsed: 1200 ms` |

Troubleshooting Tips

- **No `[DIST]` output after arming** — check HC-SR04 wiring (TRIG→22, ECHO→23, VCC→5 V, GND→GND).

- **Grace period resets unexpectedly** — the sensor is getting a timeout (`pulseIn` returning 0)

or an out-of-range reading (> 400 cm); ensure the object is flat-on to the sensor face and

within ~3 m.

- **TFT shows garbled content or stays black** — verify SPI wiring and that `INITR_BLACKTAB`

matches your Fasizi module variant; check Serial for `TFT INIT FAILED`.

- **Button causes multiple transitions** — confirm the 220 Ω resistor is not on the button line

and that `INPUT_PULLUP` is active; the 50 ms debounce handles most bounce, but very noisy

buttons may need a small hardware capacitor (100 nF across the button contacts).